

The Design and Formal Analysis of RCS: A Quantum-Resilient AEAD Scheme

John G. Underhill

Quantum Resistant Cryptographic Solutions Corporation, Canada

Abstract. RCS, the Rijndael Cipher Stream, is a wide-block authenticated encryption scheme that combines the 256-bit-block member of the Rijndael family with cSHAKE-derived round keys and a KMAC-based authentication layer. This paper gives a formal, implementation-aligned analysis of the construction under explicit PRF, PRP, and EUF-CMA assumptions. We define the AEAD syntax, key and nonce requirements, operational behavior, and single-call transform interface analyzed by the security proofs. Using this model, we establish confidentiality through a sequence of hybrid games that reduce the security of the keystream generator to the PRF security of cSHAKE and the PRP security of the Rijndael-256 permutation under derived subkeys. Integrity is shown to follow from the PRF security of the cSHAKE key-derivation step together with the existential unforgeability of KMAC, which authenticates an injectively encoded transcript comprising the associated data and its length, the nonce, the ciphertext, and a final encoded length value. Combining these results with the Encrypt-then-MAC composition theorem yields IND-CCA security for the single-call AEAD interface under unique, non-overlapping counter-interval usage.

Independent cSHAKE-derived round keys remove the structural recurrences of the classical Rijndael key schedule, while the sponge-based authentication layer avoids the small-tag polynomial-MAC structure of GMAC and Poly1305. We analyze post-quantum considerations in black-box and quantum-access models, distinguishing generic Grover-type degradation from the stronger Q2 attacks known for polynomial-MAC-based AEAD modes. We also discuss generic-model key-commitment behavior, multi-user considerations, and the operational assumptions required for correctness. The analysis identifies the limits of the construction: RCS is not nonce-misuse-resistant, relies critically on nonce uniqueness and counter-interval disjointness, and its IND-CCA result applies to the specified single-call transform rather than internal chunked helpers outside the RCS specification.

Keywords: authenticated encryption · AEAD · Rijndael-256 · cSHAKE · KMAC

Contents

1	Introduction	4
1.1	Background and Motivation	4
1.2	Contributions of this Work	4
1.3	Organization of the Paper	5
2	Engineering Description of RCS	5
2.1	Interface and Parameters	5
2.2	Internal State Layout	7
2.3	Background on Rijndael-256	8
2.4	Key Expansion via cSHAKE	8
2.5	Counter Mode Encryption Layer	8
2.6	KMAC Authentication Layer	9
2.7	Background on Keccak, cSHAKE and KMAC	10
2.8	Positioning RCS Relative to AES-GCM and ChaCha20-Poly1305	11
2.9	Pseudocode Specification	11
3	Preliminaries	13
3.1	Notation	13
3.2	Underlying Primitives	14
3.3	Adversarial Model	15
4	Formal Specification of RCS	16
4.1	Syntax of the Scheme	16
4.2	Encryption and Decryption Algorithms	16
4.3	Key and Nonce Requirements	17
5	Security Definitions	18
5.1	IND-CPA and IND-CCA Experiments	18
5.2	Ciphertext Integrity (INT-CTXT)	19
5.3	Related-Key and Misuse Notions	19
5.4	Adversarial Advantages and Parameters	20
6	Provable Security Analysis	20
6.1	Confidentiality Reduction (IND-CPA)	20
6.2	Integrity Reduction (INT-CTXT)	22
6.3	AEAD Security (IND-CCA)	23
6.4	Key-Commitment Considerations	24
6.5	Multi-User Security	25
6.6	Post-Quantum Considerations	25
7	Cryptanalytic Evaluation of RCS	25
7.1	Wide-Block Rijndael Structure	26
7.2	Differential and Linear Cryptanalysis	26
7.3	Algebraic and Structural Attacks	27
7.4	Related-Key and Key-Schedule Attacks	27
7.5	The Reduced-Rounds Configuration	27
7.6	Summary of Cryptanalytic Margins	28
8	Misuse Resistance and Operational Limits	29
8.1	Nonce Reuse and its Consequences	29
8.2	Counter-Interval Overlap and Sequential Nonces	30
8.3	Scope of the Single-Call Interface	30

John G. Underhill	3
8.4 Modeling Assumptions and Out-of-Scope Attacks	30
9 Implementation Conformance and Side Channels	31
9.1 Mapping Between Model and Reference Code	31
9.2 Constant Time Behavior and Leakage	33
9.3 Random Number Generation Requirements	34
10 Conclusion	34
10.1 Summary of Results	35
10.2 Limitations and Future Work	35
References	37

1 Introduction

RCS, the *Rijndael Cipher Stream*, is a wide-block authenticated encryption construction that integrates a 256-bit Rijndael permutation with cSHAKE-based subkey generation and a KMAC authentication layer. The design seeks to provide deterministic and structurally simple authenticated encryption while maintaining strong resilience to classical and post-quantum adversaries. Unlike modes that derive both confidentiality and integrity from a single duplex state, RCS separates its encryption and authentication components, employing counter-mode encryption over a wide permutation and an independent sponge-based MAC. This modular layout simplifies analysis, enhances domain separation, and allows each component to be treated under its respective idealized model.

The RCS construction builds directly on two well-established families of primitives. The wide-block Rijndael permutation extends the AES design to a 256-bit state, enabling deeper diffusion and reducing exposure to structural attacks on reduced-round variants. The cSHAKE and KMAC functions, derived from the Keccak sponge framework, provide domain-separated key expansion and message authentication without relying on polynomial structures that are fragile under nonce misuse. Together these yield an authenticated encryption scheme with clean composition properties and conservative cryptanalytic margins.

This paper gives a formally grounded treatment of RCS security under explicit primitive assumptions and implementation invariants. The analysis provides game-based proofs of confidentiality, ciphertext integrity, and AEAD security, each aligned with standard definitions used in modern cryptographic literature. It also justifies the selected parameters through a cryptanalytic survey of differential, linear, algebraic, and related-key attacks on wide-block Rijndael and sponge-based constructions. The engineering-level description specifies the implementation profile analyzed by the proof, so that the formal model and the specified interface are aligned.

1.1 Background and Motivation

The motivation for RCS arises from the need for an authenticated encryption primitive that is simultaneously easy to analyze, domain-separated at all interfaces, and powered by primitives whose security arguments remain stable in the post-quantum era. Conventional block-cipher-based AEAD schemes, including popular polynomial modes, rely on key schedules or linear MAC structures that introduce analytical challenges or misuse vulnerabilities. Sponge-based designs address some of these limitations but often entwine encryption and authentication within a single state, complicating proofs and side-channel hardening.

RCS instead adopts a hybrid model: Rijndael-256 is used exclusively for keystream generation in counter-mode, while a separate KMAC instance authenticates the associated data and its length, the nonce, the ciphertext, and a final encoded length value derived from the number of processed bytes. The cSHAKE key schedule removes the linear recurrences that enable related-key and slide attacks against the classical AES schedule, and the wider permutation increases the difficulty of reduced-round differential and linear trails. These design choices make RCS a useful case study in integrating well understood primitives into a conservative, post-quantum-ready AEAD construction.

1.2 Contributions of this Work

This formal cryptanalysis advances the RCS analysis in five specific ways:

- **Engineering-Level Definition.** We introduce a precise operational description of RCS, including a pseudocode specification of the single-call AEAD interface. This ensures consistent interpretation across implementations and proofs.

- **Game-Based Reductions.** We present hybrid-game proofs for confidentiality (IND-CPA), integrity (INT-CTXT), and composed authenticated encryption security (IND-CCA), reducing these properties to the PRP security of the wide-block Rijndael permutation and the PRF/EUF-CMA security of cSHAKE and KMAC. The integrity reduction includes an explicit injectivity lemma for the authenticated transcript encoding, ensuring that the EUF-CMA security of KMAC transfers to ciphertext integrity for the composed scheme.
- **Cryptanalytic Justification of Parameters.** We analyze the resistance of the 22-round (RCS-256) and 30-round (RCS-512) configurations against published differential, linear, algebraic, and related-key attacks, supporting conservative margins in each case.
- **Key-Commitment and Multi-User Considerations.** We give a generic-model argument for the key-committing behavior of the sponge-based authentication layer, contrast it with polynomial MACs, and summarize the multi-user security considerations of the construction.
- **Clarification of Operational Assumptions and Limits.** We document the required conditions for security, including nonce uniqueness and the disjointness of per-message counter intervals, and provide a clear explanation of the consequences of misuse, such as confidentiality failure under nonce reuse or counter-interval overlap.

1.3 Organization of the Paper

Section 2 provides a engineering-level description of RCS, including a Pseudocode specification that aligns with the reference implementation. Section 3 introduces the notation, primitive assumptions, and adversarial model used throughout the analysis. Section 4 formalizes the RCS construction within this model. Section 5 presents the security definitions for confidentiality, integrity, and authenticated encryption. Section 6 contains the game-based reductions establishing these guarantees. Section 7 evaluates the cryptanalytic strength of RCS against major attack families and discusses the rationale behind its parameters. Section 8 describes operational assumptions, limitations, and misuse considerations. Section 9 analyzes implementation conformance and side-channel aspects. Section 10 concludes with a summary of results and directions for further refinement.

2 Engineering Description of RCS

This section specifies the implementation profile of the RCS authenticated cipher stream analyzed in this paper. The presentation follows the public single-call AEAD interface and the state transitions required by the security proofs. All notations introduced here are later used in the formal specification and security analysis.

2.1 Interface and Parameters

RCS is an authenticated encryption with associated data (AEAD) construction with fixed parameter sets RCS-256 and RCS-512. Each parameter set defines the sizes of the cipher key and authentication tag, while the block size and nonce size are fixed across variants.

Key Size. RCS-256 uses a 256-bit key (`QSC_RCS256_KEY_SIZE` = 32 bytes). RCS-512 uses a 512-bit key (`QSC_RCS512_KEY_SIZE` = 64 bytes).

Block Size. The permutation operates on a fixed 256-bit block (`QSC_RCS_BLOCK_SIZE = 32` bytes).

Nonce. Each encryption requires a 256-bit nonce of length `QSC_RCS_NONCE_SIZE = 32` bytes. Nonce uniqueness for each key is mandatory.

Authentication Tag. RCS-256 outputs a 32-byte tag (`QSC_RCS256_MAC_SIZE`). RCS-512 outputs a 64-byte tag (`QSC_RCS512_MAC_SIZE`).

Associated Data. Associated data is an arbitrary byte string that is authenticated but not encrypted. The caller provides associated data via `qsc_rcs_set_associated` before calling the transform routines.

API Surface. The implementation exposes the following public functions:

- `qsc_rcs_initialize(state, keyparams, encrypt)` initializes the cipher, derives round keys and the MAC key, loads the nonce, and sets the mode.
- `qsc_rcs_set_associated(state, data, length)` optionally adds AEAD data to the authentication function after initialization and before the transform.
- `qsc_rcs_transform(state, out, in, length)` encrypts or decrypts a message of length `length` bytes and applies or verifies a tag. This single-call function is the AEAD interface analyzed in this paper.
- `qsc_rcs_extended_transform` is an internal chunked-processing helper used outside the RCS specification. It is excluded from the formal model, and the security theorems of Section 6 apply only to the single-call `qsc_rcs_transform` AEAD interface.

Table 1: RCS parameters for the two supported variants.

Parameter	RCS-256	RCS-512
Key length	32 bytes	64 bytes
Tag length	32 bytes	64 bytes
Rounds (R)	22	30
Round key size	32 bytes $\times$ ($R+1$)	32 bytes $\times$ ($R+1$)
Keccak rate (r)	136 bytes	136 bytes
Keccak capacity (c)	512 bits	512 bits
Domain string	<code>rcs256</code>	<code>rcs512</code>

Reduced-Rounds Build Option. The reference implementation supports an optional reduced-rounds configuration, selected by the compile-time macro `QSC_RCS_REDUCED_ROUNDS`, intended for performance-constrained deployments. In this configuration the Rijndael-256 round counts are lowered to 14 rounds for RCS-256 and 21 rounds for RCS-512, and the authentication component is switched from the standard KMAC mode (KMAC-R24, using the full 24-round Keccak- $f[1600]$ permutation) to a reduced-round mode, KMAC-R12, in which each permutation call applies 12 rounds. The KMAC-R12 instance is initialized with its own distinct name string (`KMACR12`), domain-separating it from standard KMAC; the cSHAKE key expansion itself is unchanged and always uses the full 24-round permutation. The standard and reduced builds are distinct, non-interoperable cipher configurations: they derive different numbers of round keys and produce different ciphertexts and tags for the same inputs.

The design rationale is that RCS does not rely on the classical Rijndael key schedule whose structural recurrences enable many reduced-round and related-key techniques against AES-style expansions; the cSHAKE-derived independent round keys remove that attack surface (Section 7.4), and the Keccak permutation retains a conservative margin at 12 rounds for keyed-MAC usage. The reduced-rounds option is offered as an explicit performance and security tradeoff for scenarios in which tags are verified immediately after transit and long-term cryptanalytic exposure is not the primary design goal. *The formal security claims and cryptanalytic margins established in this paper apply to the standard configuration only*; the reduced configuration is assessed separately in Section 7.5, and deployments requiring maximal conservatism or long-term confidentiality should use the standard configuration.

Table 2: Parameters of the optional reduced-rounds build (QSC_RCS_REDUCED_ROUNDS). Key, tag, block, and nonce sizes are unchanged from Table 1.

Parameter (reduced build)	RCS-256	RCS-512
Rounds (R)	14	21
Round key size	32 bytes $\times$ ($R+1$)	32 bytes $\times$ ($R+1$)
MAC mode	KMAC-R12	KMAC-R12
Keccak rounds per permutation	12	12
MAC domain string	KMACR12	KMACR12

2.2 Internal State Layout

The internal state structure `qsc_rcs_state` contains all data needed for the permutation, key schedule, counter generation, and authentication:

- **Cipher Type** (`ctype`) distinguishing RCS-256 or RCS-512.
- **Round Key Array** An array of 128-bit or 32-bit elements (depending on platform), containing the expanded subkeys for all rounds. The total number of rounds is:

$$R_{256} = 22, \quad R_{512} = 30.$$

- **Keccak State** (`kstate`) The internal sponge state used for key expansion and MAC key extraction.
- **Nonce Storage** A 32-byte buffer that is loaded with the initialization nonce and then advanced in place by the counter-mode layer; it always holds the next counter block to be encrypted.
- **Byte Counter** A 64 bit counter, initialized to the value 1 by `qsc_rcs_initialize`, that accumulates the total number of message bytes processed by the transform routines. It is used only in the final length encoding absorbed by the MAC and does not drive the counter values used by the Rijndael-256 encryption layer.
- **Mode Bit** (`encrypt`) Determines whether the transform function performs encryption or decryption.

The 256-bit Rijndael state is represented mathematically as a 4×8 byte matrix. Optimized code may hold the state in two 128-bit machine lanes, but the lanes are intermixed by the Rijndael-256 ShiftRows operation and are not independent AES states. Round keys are 256-bit values applied to the full state each round.

2.3 Background on Rijndael–256

Rijndael–256 is the 256-bit-block member of the Rijndael family. Its state is a 4×8 byte matrix, equivalently eight four-byte columns, with bytes serialized column-first. The round function applies the usual Rijndael operations over this wider state: **SubBytes** over all 32 state bytes, **ShiftRows** with the Rijndael-256 offsets $\{0, 1, 3, 4\}$, **MixColumns** independently to each of the eight columns, and **AddRoundKey** with a 256-bit round key.

In RCS, this permutation is used exclusively in counter-mode. Each keystream block is produced by applying the Rijndael–256 permutation to a 256-bit counter value, which is incremented modulo 2^{256} at each block. The master key does not feed the classical Rijndael key schedule; independently distributed round keys are derived from a cSHAKE-based expansion. Under the assumption that these round keys behave as independent 256-bit values, the permutation is modeled as a pseudo-random permutation (PRP) in the security analysis.

The reference implementation uses a scalar path that realizes the 4×8 Rijndael-256 transformation directly. Optimized AES-NI and vectorized paths may represent the same 32-byte state as two 128-bit machine lanes for instruction-set efficiency. This is an implementation representation, not a decomposition into two independent AES states. The explicit inter-lane shuffle implements the Rijndael-256 **ShiftRows** operation across the 128-bit lane boundary; after that shuffle, AES round instructions are used to perform the lane-local S-box, row, column, and round-key operations efficiently.

The RCS–256 and RCS–512 variants use 22 and 30 rounds, respectively. These round counts place the construction outside the range of currently published attacks on reduced-round large-block Rijndael variants under the assumptions used in this paper.

2.4 Key Expansion via cSHAKE

Key expansion is performed using the cSHAKE extendable-output function. The initialization absorbs:

1. the input cipher key,
2. the optional user-defined information (tweak) string `info`,
3. a fixed, scheme-specific domain string:
 - `rsc256_name` for RCS–256,
 - `rsc512_name` for RCS–512.

Once initialized, the cSHAKE instance produces a stream of pseudo-random bytes. This stream is used as follows:

- first, a contiguous block of bytes is squeezed and mapped into the array of $(R + 1)$ 256-bit round keys; any unused bytes remaining in the final squeezed rate block are discarded,
- then, a further squeeze call — which applies at least one additional Keccak permutation to the sponge state — yields the MAC key (32 bytes in RCS–256 or 64 bytes in RCS–512). The MAC key therefore never overlaps the round-key output, and its extraction is *state-separated* from the round-key extraction by the intervening permutation, matching the behavior of the CEX reference implementation.

The domain strings guarantee that RCS key expansion is isolated from other cSHAKE applications and is resistant to related-key and cross-domain collisions.

2.5 Counter Mode Encryption Layer

Encryption uses the wide-block Rijndael permutation in counter-mode.

Counter Initialization. The initial counter block is the 32-byte nonce interpreted as a 256-bit little-endian integer. For block index i , the counter value is $\text{CTR}_i = (N+i) \bmod 2^{256}$, and this value is used directly as the permutation input for that block.

keystream Generation. For each block index i :

1. Form the counter block CTR_i as

$$\text{CTR}_i = (N + i) \bmod 2^{256}.$$

2. Apply the Rijndael-256 round function with the expanded round keys:

$$KS_i = E_{RK}(\text{CTR}_i).$$

3. Increment the counter.

In the reference implementation the counter blocks are realized by treating the 32 byte nonce buffer as a 256 bit little endian integer and incrementing it in place after each block with `qsc_intutils_le8increment`; the byte counter `ctx.counter` is used only by the MAC layer.

Counter-Interval Disjointness and Parallel Processing. Because the nonce *is* the initial counter value, each message encrypted under a given key consumes the counter interval $[N, N + \lceil |P|/32 \rceil]$. Confidentiality requires that the intervals consumed by distinct messages under the same key be pairwise *disjoint*; nonce uniqueness alone is not sufficient, since two distinct nonces whose intervals overlap (for example, N and $N + 1$ for a multi-block message) would cause keystream reuse. The full-width exposed counter is a deliberate design choice: its primary purpose is to allow very large data sets to be encrypted in parallel, with worker instances assigned nonces offset at the correct block boundaries so that their counter intervals partition the data without overlap. Outside of such deliberately partitioned parallel deployments, the nonce must be unique *and random* for each encryption cycle, generated by a CSPRNG; for random 256-bit nonces the probability of an accidental interval overlap is negligible (Section 6.1). Naive sequential nonces $(N, N + 1, N + 2, \dots)$ must *not* be used; see Section 8.2.

Vectorized Counter Path. The AVX-512 code path in the reference implementation increments only the low 64-bit lane of the counter for performance, providing 2^{64} keystream blocks (2^{69} bytes) per message before lane rollover. Its output is identical to the carry-propagating scalar and AES-NI paths whenever the low 64-bit lane does not wrap within the call, a precondition that holds except with negligible probability for random nonces; Section 9.2 specifies this precondition and a guard that enforces it.

Encryption and Decryption. The keystream block KS_i is XORed with the plaintext block P_i to form ciphertext C_i , and conversely during decryption.

2.6 KMAC Authentication Layer

RCS authenticates ciphertext using a KMAC instance whose key is derived from the same cSHAKE instance as the round keys, but extracted from a subsequent, state-separated squeeze of the sponge (Section 2.4). Domain separation in the SP 800-185 sense is provided at the cSHAKE level by the `rscs256/rscs512` function-name strings; the KMAC instance itself is keyed with the derived MAC key and uses no additional customization string.

The authenticated input to KMAC consists of a deterministic transcript that includes:

- the associated data AD (when present),
- a fixed-width 32-bit encoding of $|AD|$ (when present),
- the 32-byte nonce N as loaded at initialization,
- the ciphertext C , and
- a final fixed-width 64-bit encoding whose value is $\text{QSC_RCS_BLOCK_SIZE} + \text{ctr} + 8$, where ctr is the byte counter (initialized to 1 and incremented by the message length); for a single-call transform this value equals $41 + |C|$ with $|C|$ in bytes.

Tag Generation. Upon finalizing encryption:

1. all associated data is absorbed, followed by a fixed-width encoding of its length, via `qsc_rcs_set_associated`,
2. immediately before processing a message chunk, the current 32-byte nonce value is absorbed (for the single-call interface this is exactly the initialization nonce N , since the counter-mode layer has not yet advanced the nonce buffer),
3. each ciphertext chunk is absorbed during the transform, and the running byte counter is updated,
4. at finalization, the fixed-width 64-bit length encoding defined above is absorbed, and
5. a tag of length 32 bytes in RCS-256 or 64 bytes in RCS-512 is output.

Tag Verification. During decryption with the single-call `qsc_rcs_transform` interface, a tag is recomputed and must match the provided tag in constant time. If verification fails, decryption returns failure and no plaintext is released. Internal chunked helpers are outside the RCS formal specification.

Table 3: Components of the authenticated transcript absorbed by KMAC (single-call interface).

Component	Description
AD	Associated data (absorbed once, if present)
<code>encode_len(AD)</code>	32-bit little-endian encoding of AD length
N	32-byte nonce, absorbed before the counter advances
C	Ciphertext bytes
<code>encode_len($41 + C$)</code>	64-bit little-endian final length encoding

2.7 Background on Keccak, cSHAKE and KMAC

RCS uses two components from the Keccak family: cSHAKE for key expansion and KMAC for message authentication. Both are derived from the Keccak sponge construction, which absorbs input blocks into a large permutation state and later squeezes output blocks from the same state. The permutation used here is Keccak- $f[1600]$, which updates a 1600-bit internal state through 24 rounds of nonlinear and linear mixing.

The sponge construction is parameterized by a rate r and capacity c , with $r + c = 1600$. RCS uses the 256-bit security configuration, giving a rate of $r = 1088$ bits (136 bytes) and a capacity of $c = 512$ bits. Each call to the squeeze function yields up to 136 bytes before requiring another permutation of the Keccak state.

The cSHAKE variant provides domain separation through a function-name string and an optional customization string. In RCS, the master key, the optional user-defined `info`

value and a fixed function-name string (`rsc256` or `rsc512`) are absorbed to initialize the state. The output stream is interpreted as a sequence of pseudo-random bytes from which RCS slices $(R + 1)$ independent 256-bit round keys. The MAC key is then obtained from a further squeeze call, separated from the round-key material by at least one additional application of the Keccak permutation; under the XOF model all output positions are independent, and this state separation ensures the MAC key never overlaps the round-key output.

KMAC is a message authentication code built on cSHAKE, using a sponge-based absorb-then-squeeze interface. In RCS, KMAC authenticates a transcript that includes the associated data and its length encoding, the 32-byte nonce, the ciphertext, and a final encoding of the total processed-byte count. Under the standard PRF and EUF-CMA assumptions for sponge-based MACs with capacity $c = 512$, this construction provides strong binding of all inputs to the tag.

2.8 Positioning RCS Relative to AES-GCM and ChaCha20-Poly1305

Authenticated encryption in current network protocols is dominated by AES-GCM and ChaCha20-Poly1305. Both are efficient, mature, and widely deployed AEAD schemes. RCS is not presented as a drop-in replacement for all such deployments; it is a conservative alternative for settings that prioritize larger generic authentication margins, wide-block counter-mode encryption, and sponge-based key derivation and authentication.

The main structural distinction lies in the authentication layer. GMAC and Poly1305 are polynomial MACs with 128-bit tags. In the classical setting this limits the generic authentication margin to 128 bits, and in a black-box quantum search model the corresponding brute-force tag margin is reduced to roughly 64 bits. Stronger quantum-access results are also known for polynomial-MAC-based AEAD modes: GMAC/GCM is insecure in the Q2 superposition-query model by Simon-period-finding attacks, and Poly1305 has published Q2 attacks with complexities far below its classical 128-bit target. These results do not make AES-GCM or ChaCha20-Poly1305 obsolete in ordinary classical deployments, but they show that 128-bit polynomial-MAC authentication is not a conservative choice for high-assurance post-quantum authentication targets.

RCS replaces the polynomial MAC with KMAC, a Keccak sponge based authenticator, and uses 32-byte or 64-byte authentication tags depending on the parameter set. The cSHAKE key expansion produces domain-separated round keys and a MAC key without relying on the linear recurrence of the classical Rijndael key schedule. The encryption layer is a wide-block Rijndael permutation used in counter-mode, while the authentication layer binds the associated data, nonce, ciphertext, and length encodings through a keyed sponge transcript.

Operationally, RCS also preserves some implementation continuity with AES-family software and hardware techniques. Optimized implementations can exploit AES-NI or vector instructions for the Rijndael round operations while adding the Rijndael-256 inter-lane shuffle required by the eight-column state. The resulting design trades the standardization and deployment maturity of mainstream AEADs for a larger tag space, a sponge-based authenticator, and a wide-block counter-mode structure intended for high-assurance post-quantum-transition environments.

2.9 Pseudocode Specification

The following Pseudocode mirrors the behavior of `qsc_rsc_initialize` and `qsc_rsc_transform`.

Algorithm 1 RCS_INITIALIZE**Require:** Context structure `ctx`, key parameters `params`, flag `encrypt`**Ensure:** Initialized RCS state in `ctx`

```

1: // Load key parameters
2:  $K \leftarrow \text{params.key}$ 
3:  $\text{info} \leftarrow \text{params.info}$ 
4:  $N \leftarrow \text{params.nonce}$ 
5: // Initialize context
6:  $\text{ctx.encrypt} \leftarrow \text{encrypt}$ 
7:  $\text{ctx.nonce} \leftarrow N$ 
8:  $\text{ctx.counter} \leftarrow 1$ 
9:  $\text{ctx.ctype} \leftarrow \text{params.ctype}$ 
10: // Initialize cSHAKE with key, domain string, and optional info
11:  $S \leftarrow \text{CSHAKE\_INIT}(K, \text{rcs\_name}, \text{info})$ 
12: // Derive round keys as a single squeezed byte stream
13:  $\text{rk\_bytes} \leftarrow \text{SPONGE\_SQUEEZE}(S, (R + 1) \cdot 32)$ 
14: for  $i = 0$  to  $R$  do
15:    $\text{ctx.RK}[i] \leftarrow \text{SLICE}(\text{rk\_bytes}, 32 \cdot i, 32)$ 
16: end for
17: // Derive MAC key from a further squeeze; the intervening permutation
18: // state-separates the MAC key from the round-key material
19:  $\text{ctx.mac\_key} \leftarrow \text{SPONGE\_SQUEEZE}(S, \text{MAC\_LEN})$ 
20: // Initialize KMAC state
21:  $\text{KMAC\_INIT}(\text{ctx.mac}, \text{ctx.mac\_key})$ 

```

$\text{SpongeSqueeze}(S, \text{rlen})$ denotes a contiguous squeeze of rlen bytes from the cSHAKE sponge, implemented internally by repeated calls to the rate-sized block function as in the reference code.

Algorithm 2 RCS_TRANSFORM (single-call AEAD interface)**Require:** Context structure `ctx`, output buffer `output`, input buffer `input`, length `length`**Ensure:** On success, `output` contains ciphertext or plaintext and the function returns `true`

```

1: // Update processed byte counter; counter was initialized to 1
2: ctx.counter  $\leftarrow$  ctx.counter + length
3: // Absorb the nonce; the counter-mode layer has not yet advanced it
4: KMAC_UPDATE(ctx.mac, ctx.nonce)
5: // Set the expected tag length from the cipher type
6: if ctx.ctype = RCS256 then
7:   maclen  $\leftarrow$  32
8: else
9:   maclen  $\leftarrow$  64
10: end if
11: if ctx.encrypt = true then
12:   // Encryption: transform, absorb ciphertext, finalize tag
13:   RCS_CTR_TRANSFORM(ctx, output, input, length)
14:   KMAC_UPDATE(ctx.mac, output[0..length-1])
15:   KMAC_UPDATE(ctx.mac, ENCODE_LEN(32 + ctx.counter + 8))
16:   tag  $\leftarrow$  KMAC_FINAL(ctx.mac)
17:   copy tag into output[length..length+maclen-1]
18:   return true
19: else
20:   // Decryption: absorb ciphertext, verify tag, decrypt only on success
21:   KMAC_UPDATE(ctx.mac, input[0..length-1])
22:   KMAC_UPDATE(ctx.mac, ENCODE_LEN(32 + ctx.counter + 8))
23:   code  $\leftarrow$  KMAC_FINAL(ctx.mac)
24:   ok  $\leftarrow$  VERIFYEQUAL(code, input[length..length + maclen - 1])
25:   if ok  $\neq$  true then
26:     // MAC check failed; no plaintext is produced
27:     return false
28:   end if
29:   // Only reached when the MAC check has passed
30:   RCS_CTR_TRANSFORM(ctx, output, input, length)
31:   return true
32: end if

```

The internal `qsc_rcs_extended_transform` helper is a chunked-processing routine used outside the RCS specification. It is excluded from the formal AEAD model and from the theorems of Section 6.

3 Preliminaries

This section introduces the notation, idealized models, and adversarial framework used throughout the security analysis. All definitions follow standard conventions in symmetric cryptography and AEAD security proofs.

3.1 Notation

A bit string X of length n is written $X \in \{0,1\}^n$, and $|X|$ denotes its length in bits. Concatenation is written $X \parallel Y$. The bitwise exclusive-or operator is denoted $X \oplus Y$. For

a positive integer q , we write $[q] = \{1, 2, \dots, q\}$.

Random Sampling. The notation $x \xleftarrow{\$} S$ denotes sampling a value x uniformly at random from a finite set S . The notation $x \xleftarrow{\$} \{0, 1\}^n$ denotes choosing a uniform n -bit string.

Probability. For a probabilistic experiment Exp and event E , the probability that E occurs is written $\Pr[\text{Exp} \Rightarrow E]$. All probabilities are taken over the randomness of the experiment and the adversary.

Negligible Functions. A function $\mu(\lambda)$ is negligible in the security parameter λ if for every polynomial $p(\cdot)$ there exists N such that $\mu(\lambda) < 1/p(\lambda)$ for all $\lambda > N$.

Encoding. The MAC input uses fixed width little endian encodings. The associated data length is encoded as a 32 bit value, and the final length field is encoded as a 64 bit value. We write `encode_len( $\cdot$ )` for this fixed width encoding, with the understanding that the width depends on the field (32 bits for associated data length, 64 bits for the final field). The value of the final 64-bit field is $32 + \text{ctr} + 8$, where `ctr` is the byte counter initialized to 1 and incremented by the number of message bytes processed; for the single-call interface this equals $41 + |C|$ with $|C|$ the ciphertext length in bytes. The final field is therefore a fixed affine function of $|C|$ and uniquely determines it.

3.2 Underlying Primitives

The RCS construction relies on three cryptographic components: a wide-block Rijndael permutation, a cSHAKE-based key expansion function, and a KMAC-based message authentication function. Each is analyzed under a standard idealized model.

Rijndael-256 as a PRP. The wide-block Rijndael permutation is modeled as a pseudo-random permutation (PRP) over 256-bit blocks under independent round keys. Formally, for a key K and input $X \in \{0, 1\}^{256}$, the function $E_K(X)$ is assumed to be indistinguishable from a uniformly chosen permutation over $\{0, 1\}^{256}$ when keyed with independently distributed subkeys. This idealization is used in the confidentiality proof.

cSHAKE as a PRF. The key expansion function is modeled as a pseudo random function (PRF) from the master key, an optional info string, and a domain separated customization string to an unbounded pseudo-random output stream. Given the indistinguishability of the Keccak sponge from a random oracle, and the domain separation of the RCS customization strings, this model captures the claimed behavior of cSHAKE for key derivation and round-key expansion.

KMAC as a PRF and EUF-CMA MAC. The KMAC instance used by RCS is modeled both as:

1. a PRF keyed by a cSHAKE-derived MAC key for the purposes of confidentiality proofs involving hybrid transitions, and
2. an existentially unforgeable MAC (EUF-CMA) for the integrity and AEAD proofs.

The MAC key is derived from the same domain-separated cSHAKE instance as the round keys, but from a subsequent, state-separated squeeze of the sponge (Section 2.4); under the PRF model for cSHAKE, the MAC key is therefore independent of the round

keys generated during key expansion. The KMAC instance itself is keyed with this derived MAC key and uses no additional customization string.

Sponge and Random-Oracle Idealization. Where appropriate, we use the indistinguishability of Keccak’s sponge construction from a random oracle to justify the PRF behavior of both cSHAKE and KMAC, following the standard security arguments for Keccak-family XOFs.

3.3 Adversarial Model

We consider probabilistic polynomial-time (PPT) adversaries interacting with the RCS construction through oracle interfaces defined by the standard AEAD security experiments. Unless stated otherwise, all adversaries operate in the classical query model. Quantum-query considerations are addressed separately in the post-quantum subsection of the security analysis.

Adversary Classes. Throughout this work, we consider:

- IND-CPA adversaries with access to an encryption oracle,
- IND-CCA adversaries with access to both encryption and decryption oracles, where decryption returns $\perp$ on invalid tags,
- INT-CTXT adversaries attempting to forge a ciphertext–tag pair,
- adversaries attempting to exploit related-key or structural properties, modeled under the PRF idealization of cSHAKE.

Query Bounds. Let q_E and q_D denote the number of encryption and decryption queries, respectively, let ℓ denote the maximum total message length across all queries, and let $\sigma = \lceil \ell/256 \rceil$ denote the corresponding total number of 256-bit keystream blocks. Advantage bounds throughout the proofs are expressed as functions of (q_E, q_D, ℓ, σ) and the security parameter $\lambda \in \{256, 512\}$.

Oracle Semantics. The encryption oracle returns ciphertext and tag pairs produced by the real algorithm, while the decryption oracle outputs the decrypted plaintext if and only if the supplied tag is correct. Outputs are suppressed on tag failure, preventing side-channel oracles that could bias the adversary.

Nonce Conditions. All security definitions assume that the adversary is *interval-respecting*: nonces are never repeated for a given key, and in addition the counter intervals $[N_j, N_j + \sigma_j)$ consumed by distinct encryption queries are pairwise disjoint, where σ_j is the number of keystream blocks of query j and interval arithmetic is modulo 2^{256} . Nonce uniqueness alone is insufficient in a nonce-as-counter design, since overlapping intervals cause keystream reuse even under distinct nonces. Operationally, interval disjointness is achieved either by sampling nonces uniformly at random (in which case an overlap occurs only with the negligible probability bounded in Section 6.1) or, in deliberately partitioned parallel deployments, by assigning nonces offset at block boundaries so that the intervals partition the data. The security model does not provide guarantees under nonce reuse or interval overlap, and these limitations are explicitly documented in the misuse analysis.

Post-Quantum Considerations. We adopt the standard square-root degradation model for symmetric primitives under quantum adversaries; reductions in the post-quantum analysis quantify the expected security loss in terms of $\sqrt{2^\lambda}$ generic search bounds.

4 Formal Specification of RCS

This section defines the RCS authenticated encryption scheme in mathematical terms. The description abstracts the engineering-level construction of Section 2 into a concise formal model suitable for security analysis.

Let $\lambda \in \{256, 512\}$ denote the security parameter corresponding to the key size. All bit strings and operations follow the notation established in the Preliminaries.

4.1 Syntax of the Scheme

RCS is an authenticated encryption with associated data (AEAD) scheme consisting of three algorithms:

$$\text{RCS} = (\text{KeyGen}, \text{Enc}, \text{Dec}).$$

Key Generation. A key K is a uniform bit string of length λ . Nonces are 256-bit strings and are supplied externally by the caller. RCS does not generate nonces internally.

$$K \xleftarrow{\$} \{0, 1\}^\lambda, \quad N \in \{0, 1\}^{256}.$$

Encryption. Given a key K , nonce N , associated data $A \in \{0, 1\}^*$, and plaintext $P \in \{0, 1\}^*$, the encryption algorithm outputs a ciphertext C and tag T :

$$(C, T) \leftarrow \text{Enc}(K, N, A, P).$$

Decryption. Given (K, N, A, C, T) , the decryption algorithm outputs either the plaintext P or the failure symbol $\perp$:

$$P \leftarrow \text{Dec}(K, N, A, C, T).$$

Decryption returns P if and only if the tag verifies correctly.

4.2 Encryption and Decryption Algorithms

We describe the algorithms using compact mathematical definitions that correspond to the Pseudocode in Section 2.9.

Round-Key and MAC-Key Derivation. Let:

$$(RK_0, RK_1, \dots, RK_R) \leftarrow \text{cSHAKE}(K, \text{info}, d_{\text{RCS}})$$

denote the cSHAKE output stream, domain separated by d_{RCS} , partitioned into $(R + 1)$ independent 256-bit round keys followed by a MAC key, where the MAC key is obtained only after at least one additional permutation step on the sponge state. The number of rounds is:

$$R = \begin{cases} 22 & \text{for RCS-256,} \\ 30 & \text{for RCS-512.} \end{cases}$$

Counter-Mode Encryption. Define the initial counter block $CTR_0 = N$, and for each $i \geq 0$ let $CTR_i = (N + i) \bmod 2^{256}$, where the addition is on the 256-bit little-endian representation of N . For each block P_i of the plaintext,

$$KS_i = E_{RK}(CTR_i), \quad C_i = P_i \oplus KS_i,$$

where E_{RK} is the Rijndael-256 round function using round keys $(RK_0, \dots, RK_R)$.

The ciphertext C is the concatenation of all C_i .

Tag Generation. Let $\ell = |P|$ denote the plaintext length in bytes. The authentication tag is generated via KMAC over a deterministic transcript that includes A , its encoded length, the nonce N , the ciphertext C , and the final encoded length field. Concretely, define

$$\tau(A, N, C) = \begin{cases} A \parallel \text{le}_{32}(|A|) \parallel N \parallel C \parallel \text{le}_{64}(41 + |C|) & \text{if } |A| > 0, \\ N \parallel C \parallel \text{le}_{64}(41 + |C|) & \text{if } |A| = 0, \end{cases}$$

where le_{32} and le_{64} denote the fixed-width little-endian encodings of Section 3.1, lengths are in bytes, and the constant $41 = 32 + 8 + 1$ reflects the nonce block size, the width of the length field, and the initialization value of the byte counter. The tag is

$$T = \text{KMAC}(\text{mkey}, \tau(A, N, C)),$$

and $\tau(A, N, C)$ corresponds exactly to the absorption sequence of the engineering description in Section 2.6; the conformance of the reference implementation to this transcript is established in Section 9.1.

Full Encryption Algorithm. In summary,

$$\text{Enc}(K, N, A, P) = (C, T)$$

where C and T are computed as above.

Decryption Algorithm. To decrypt (C, T) :

1. Recompute the MAC key mkey and round keys via the same cSHAKE call.
2. Compute

$$T' = \text{KMAC}(\text{mkey}, \tau(A, N, C)),$$

where $\tau(A, N, C)$ is the deterministic, injectively encoded transcript defined above, consisting of the associated data and its fixed width length encoding (when present), the 32 byte nonce, the ciphertext, and the final encoded length field.

3. If $T' \neq T$, return $\perp$.
4. Otherwise, derive $KS_i = E_{RK}(CTR_i)$ and recover $P_i = C_i \oplus KS_i$ for each block.

Correctness follows from the fact that both encryption and decryption use identical counter values and identical keystream blocks.

4.3 Key and Nonce Requirements

RCS imposes the following requirements on its inputs:

- **Key Length.** Keys must be uniformly sampled from $\{0, 1\}^{256}$ or $\{0, 1\}^{512}$.
- **Nonce Length, Uniqueness, and Interval Disjointness.** The nonce must be a 256-bit string and *must not* repeat for a given key. Because the nonce is the initial counter value, the counter intervals $[N, N + \lceil |P|/32 \rceil)$ consumed by distinct messages under the same key must additionally be pairwise disjoint. Nonces must be generated uniformly at random by a CSPRNG, except in deliberately partitioned parallel deployments where nonces are offset at block boundaries to produce disjoint intervals by construction. RCS provides no security guarantees under nonce reuse or counter-interval overlap.
- **Tag Length.** The tag length is fixed by the parameter set:

$$\text{taglen} = \begin{cases} 32 \text{ bytes} & (\text{RCS-256}), \\ 64 \text{ bytes} & (\text{RCS-512}). \end{cases}$$

- **Associated Data.** Any string supplied as A is authenticated but not encrypted and must be provided in full before encryption or decryption proceeds.

These conditions reflect the operational requirements of the reference implementation and define the scope in which the security proofs apply.

5 Security Definitions

This section defines the confidentiality and integrity notions used in the analysis of RCS. All definitions follow standard AEAD security frameworks, including the IND-CPA, IND-CCA, and INT-CTXT notions. These definitions form the basis for the game-based proofs developed in Section 6.

Let $\lambda \in \{256, 512\}$ denote the security parameter corresponding to the key size. All adversaries are probabilistic polynomial-time (PPT) algorithms unless otherwise stated.

5.1 IND-CPA and IND-CCA Experiments

Confidentiality is expressed through chosen-plaintext (IND-CPA) and chosen-ciphertext (IND-CCA) indistinguishability experiments. Each experiment allows the adversary to interact with oracles that implement the AEAD interface of RCS.

IND-CPA Experiment. The IND-CPA experiment $\text{Exp}_{RCS}^{\text{ind-cpa}}(A)$ proceeds:

1. A key $K \xleftarrow{\$} \{0, 1\}^\lambda$ is sampled.
2. The adversary A may adaptively query an encryption oracle:

$$\mathcal{O}_{\text{enc}}(A, P_0, P_1, N)$$

where P_0, P_1 are equal-length plaintexts and N is a nonce chosen by A .

3. A hidden bit $b \xleftarrow{\$} \{0, 1\}$ is sampled.
4. The oracle returns

$$(C, T) \leftarrow \text{Enc}(K, N, A_{\text{assoc}}, P_b),$$

where A_{assoc} denotes the associated data supplied by A .

5. A outputs a bit b' .

The adversary's advantage is:

$$\text{Adv}_{RCS}^{\text{ind-cpa}}(A) = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

IND-CCA Experiment. The IND-CCA experiment extends the IND-CPA experiment by allowing access to a decryption oracle, with tag-verification failures suppressed:

1. A key K is generated as above.
2. The encryption oracle is identical to the IND-CPA experiment.
3. The adversary may also query a decryption oracle:

$$\mathcal{O}_{\text{dec}}(A, N, A_{\text{assoc}}, C, T),$$

which returns $\text{Dec}(K, N, A_{\text{assoc}}, C, T)$ unless (C, T) was previously returned by the encryption oracle.

4. Adversary outputs a guess b' .

The IND-CCA advantage is:

$$\text{Adv}_{RCS}^{\text{ind-cca}}(A) = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

These definitions assume an interval-respecting adversary (Section 3.3): nonces never repeat under the same key, and the counter intervals consumed by distinct encryption queries are pairwise disjoint. Counter-mode encryption provides no confidentiality under nonce reuse, and in a nonce-as-counter design overlapping intervals cause keystream reuse even under distinct nonces.

5.2 Ciphertext Integrity (INT-CTXT)

Ciphertext integrity captures the adversary's inability to produce a valid ciphertext-tag pair not previously output by the encryption oracle. The INT-CTXT experiment $\text{Exp}_{RCS}^{\text{int-ctxt}}(A)$ is defined:

1. A key K is chosen uniformly at random.
2. A may query an encryption oracle $\mathcal{O}_{\text{enc}}(A, P, N)$, receiving (C, T) for chosen (P, N) .
3. Eventually A outputs a tuple (N^*, A^*, C^*, T^*) .
4. A wins if:
 - (a) (C^*, T^*) was not the output of any encryption query,
 - (b) $\text{Dec}(K, N^*, A^*, C^*, T^*) \neq \perp$.

The integrity advantage is:

$$\text{Adv}_{RCS}^{\text{int-ctxt}}(A) = \Pr[\text{Exp}_{RCS}^{\text{int-ctxt}}(A) = 1].$$

This definition captures forgeries, truncation attempts, block reordering, and bit-flip attacks, since any modification of C results in tag failure with all but negligible probability under the EUF-CMA security of KMAC.

5.3 Related-Key and Misuse Notions

RCS uses cSHAKE to derive independently distributed round keys from the master key, the optional info string, and a fixed domain-separating function-name string. This prevents the algebraic and linear relations exploited by classical related-key attacks on AES-like key schedules.

Related-Key Security. For completeness, we consider adversaries whose queries may involve keys drawn from a related-key oracle:

$$K' = f(K),$$

where f is an efficiently computable key-derivation function chosen by the adversary.

In the RCS model, all round keys and MAC keys are derived from cSHAKE under independent domain-separated inputs; under the PRF assumption for cSHAKE, related-key attacks reduce to distinguishing cSHAKE from a random oracle, and are therefore out of scope for the permutation-based part of the cipher.

Misuse Considerations. This work does *not* analyze RCS under nonce reuse or nonce-misuse scenarios. In counter-mode, reuse of a nonce immediately leads to keystream reuse, enabling trivial disclosure of plaintext relations:

$$C_1 \oplus C_2 = P_1 \oplus P_2.$$

Thus, RCS is *not* an MRAE (misuse-resistant authenticated encryption) scheme. Nonce uniqueness is mandatory for confidentiality guarantees.

5.4 Adversarial Advantages and Parameters

Security bounds in the proofs are expressed using standard advantage notation:

$$\text{Adv}_{RCS}^{\text{ind-cpa}}(A), \quad \text{Adv}_{RCS}^{\text{ind-cca}}(A), \quad \text{Adv}_{RCS}^{\text{int-ctxt}}(A).$$

These depend on the following parameters:

- q_E : number of encryption queries,
- q_D : number of decryption queries (IND-CCA only),
- ℓ : total length of all encrypted messages,
- σ : total number of 256-bit keystream blocks across all queries,
- λ : security parameter (256 or 512),
- R : number of rounds (22 or 30),
- taglen : tag length (32 or 64 bytes).

All advantages are functions of $(q_E, q_D, \ell, \sigma, \lambda)$ and are negligible in λ for the intended usage of RCS.

6 Provable Security Analysis

In this section we relate the security of RCS to the assumed security of its underlying components: the Rijndael-256 permutation, the cSHAKE-based key expansion, and the KMAC-based authentication layer. All adversaries are probabilistic polynomial time unless otherwise specified, and all advantages are defined as in Section 5.

6.1 Confidentiality Reduction (IND-CPA)

We prove that RCS achieves IND CPA confidentiality for interval-respecting adversaries (Section 3.3), assuming that cSHAKE behaves as a PRF and Rijndael-256 behaves as a PRP under independent round keys. The proof is given as a sequence of hybrid games.

Theorem 6.1 (IND CPA security of RCS). Let A be any interval-respecting IND CPA adversary against RCS that makes at most q_E encryption queries whose total length corresponds to at most σ keystream blocks. Then there exist adversaries B_{prf} and B_{prp} such that

$$\text{Adv}_{RCS}^{\text{ind-cpa}}(A) \leq \text{Adv}_{\text{cSHAKE}}^{\text{prf}}(B_{\text{prf}}) + \text{Adv}_{\text{Rijndael}}^{\text{prp}}(B_{\text{prp}}) + \frac{\sigma^2}{2^{257}}.$$

If, instead of being interval-respecting by construction, the nonces are sampled uniformly at random by the challenger, the bound holds with the additional term $q_E \sigma / 2^{256}$, which upper-bounds the probability that two randomly placed counter intervals overlap.

Proof. We consider a sequence of hybrids $H_0, \dots, H_3$ and bound the distinguishing advantage between successive games.

Game H_0 (Real RCS). This is the real IND CPA experiment for RCS. The key K is sampled uniformly. For each encryption query (N, A, P_0, P_1) of equal-length messages, the challenger samples a hidden bit b , derives the MAC key and round keys from cSHAKE, derives keystream blocks by applying the Rijndael-256 permutation in counter-mode, and returns the ciphertext and tag for P_b .

By definition,

$$\text{Adv}_{RCS}^{\text{ind-cpa}}(A) = |\Pr[H_0(A) = 1] - \frac{1}{2}|.$$

Game H_1 (Replace cSHAKE by a PRF). In H_1 , the challenger replaces the cSHAKE-based key expansion by a uniformly random tuple. The key expansion of RCS depends only on the master key K , the optional info string, and the fixed function-name string; in particular it does *not* depend on the nonce, so the same round keys and MAC key are used across all encryption queries. Concretely, at the start of the experiment the challenger samples once:

$$\text{mkey}, RK_0, \dots, RK_R \xleftarrow{\$} \{0, 1\}^{8 \cdot \text{taglen}} \times (\{0, 1\}^{256})^{R+1},$$

and uses this fixed tuple for every query, exactly as the real scheme reuses the output of the single cSHAKE derivation for the lifetime of the key.

The view of A in H_1 is identical to its view in H_0 unless A can distinguish the cSHAKE output stream from uniformly random bytes. Thus there exists an adversary B_{prf} such that

$$|\Pr[H_0(A) = 1] - \Pr[H_1(A) = 1]| \leq \text{Adv}_{\text{cSHAKE}}^{\text{prf}}(B_{\text{prf}}).$$

Note that this transition also makes mkey independent of the round keys; the tags returned alongside the ciphertexts are deterministic functions of (mkey, A, N, C) , all of whose non-key inputs are already in the adversary's view, so the tags contribute no additional distinguishing advantage in the subsequent games.

Game H_2 (Replace Rijndael by a random permutation). In H_2 , the challenger replaces the Rijndael-256 permutation E_{RK} , keyed by the single fixed round-key tuple of H_1 , by one ideal random permutation π over $\{0, 1\}^{256}$. For each block encryption, the challenger evaluates π on the current counter value and uses the result as the keystream block.

By the PRP assumption on Rijndael-256 there exists an adversary B_{prp} such that

$$|\Pr[H_1(A) = 1] - \Pr[H_2(A) = 1]| \leq \text{Adv}_{\text{Rijndael}}^{\text{prp}}(B_{\text{prp}}).$$

Game H_3 (Replace the PRP based keystream by a random function). Because A is interval-respecting, the counter intervals of distinct queries are pairwise disjoint, and within a query the counter values are distinct by construction; hence all σ permutation inputs across the experiment are distinct. (If nonces are instead sampled uniformly at random, two intervals of lengths σ_j and σ_k overlap with probability at most $(\sigma_j + \sigma_k)/2^{256}$, and a union bound over all pairs of queries gives an overlap probability of at most $q_E \sigma / 2^{256}$, which is added to the bound.)

In H_3 the challenger replaces π by a uniformly random function $\rho : \{0, 1\}^{256} \rightarrow \{0, 1\}^{256}$. By the PRP-PRF switching lemma, evaluating a random permutation on σ distinct points is indistinguishable from evaluating a random function except with probability at most

$$\binom{\sigma}{2} 2^{-256} \leq \frac{\sigma^2}{2^{257}},$$

so

$$|\Pr[H_2(A) = 1] - \Pr[H_3(A) = 1]| \leq \frac{\sigma^2}{2^{257}}.$$

In H_3 the keystream blocks are uniform and independent for each distinct counter value. As a result, for any fixed choice of b , the ciphertexts in H_3 are one time pad encryptions of P_b under fresh random keystream blocks, and the tags are functions of values independent of b given the ciphertexts. Hence A has no information about b beyond random guessing, and

$$\Pr[H_3(A) = 1] = \frac{1}{2}.$$

Bounding the overall advantage. By a telescoping sum over the four hybrids,

$$\begin{aligned} |\Pr[H_0(A) = 1] - \frac{1}{2}| &\leq |\Pr[H_0(A) = 1] - \Pr[H_1(A) = 1]| \\ &\quad + |\Pr[H_1(A) = 1] - \Pr[H_2(A) = 1]| \\ &\quad + |\Pr[H_2(A) = 1] - \Pr[H_3(A) = 1]|. \end{aligned}$$

Substituting the bounds for each transition yields the stated inequality. $\square$

6.2 Integrity Reduction (INT-CTXT)

We now show that any successful ciphertext forgery against RCS can be converted into a forgery against the underlying KMAC instance, after a preliminary hybrid that uses the PRF security of cSHAKE to decouple the MAC key from the round keys. The reduction relies on two facts: first, that after the PRF hybrid the MAC key is independent of the round keys, so the keystream can be simulated without knowledge of the MAC key; and second, that the transcript encoding is injective, so distinct AEAD inputs always produce distinct MAC inputs.

Lemma 6.2 (Transcript Injectivity). The map $(A, N, C) \mapsto \tau(A, N, C)$ defined in Section 4.2 is injective: for all $(A, N, C) \neq (A', N', C')$ with $N, N' \in \{0, 1\}^{256}$,

$$\tau(A, N, C) \neq \tau(A', N', C').$$

Proof. Suppose $\tau(A, N, C) = \tau(A', N', C') = X$ as byte strings; we show the preimages are equal. Parse X from the right. The final 8 bytes of any transcript are $\text{le}_{64}(41 + |C|)$, a fixed-width field whose value is an injective affine function of the ciphertext byte length; equality of the two transcripts therefore forces $|C| = |C'|$. The $|C|$ bytes preceding the final field are exactly the ciphertext, so $C = C'$. The 32 bytes preceding the ciphertext are the fixed-width nonce, so $N = N'$. The remaining prefix is either empty (the case $|A| = 0$) or equals $A \parallel \text{le}_{32}(|A|)$ with $|A| > 0$; since both transcripts have the same total length, the prefixes have equal length p . If $p = 0$ then $|A| = |A'| = 0$. If $p > 0$, then $p \geq 5$ in both encodings (a nonempty A plus a 4-byte length field), the last 4 bytes of the prefix encode $|A| = p - 4 = |A'|$, and the first $p - 4$ bytes are the associated data itself, so $A = A'$. In every case $(A, N, C) = (A', N', C')$. $\square$

Theorem 6.3 (INT CTXT security of RCS). Let A be any INT CTXT adversary against RCS making at most q_E encryption queries. Then there exist adversaries B_{prf} and B_{mac} such that

$$\text{Adv}_{\text{RCS}}^{\text{int-ctxt}}(A) \leq \text{Adv}_{\text{cSHAKE}}^{\text{prf}}(B_{\text{prf}}) + \text{Adv}_{\text{KMAC}}^{\text{euf-cma}}(B_{\text{mac}}),$$

where $\text{Adv}_{\text{KMAC}}^{\text{euf-cma}}$ denotes the standard existential unforgeability advantage for KMAC.

Proof. We proceed in two games.

Game G_0 (Real INT CTXT experiment). This is the real experiment: the challenger samples K uniformly, derives $(\text{mkey}, RK_0, \dots, RK_R)$ from cSHAKE, and answers encryption queries with the real algorithm. By definition, $\text{Adv}_{RCS}^{\text{int-ctxt}}(A) = \Pr[G_0(A) = 1]$.

Game G_1 (Replace cSHAKE by uniform key material). The challenger replaces the cSHAKE derivation by an independent uniform tuple $(\text{mkey}, RK_0, \dots, RK_R)$, sampled once at the start of the experiment. As in the proof of Theorem 6.1, any change in A 's success probability yields a cSHAKE PRF distinguisher, so

$$|\Pr[G_0(A) = 1] - \Pr[G_1(A) = 1]| \leq \text{Adv}_{\text{cSHAKE}}^{\text{prf}}(B_{\text{prf}}).$$

Crucially, in G_1 the MAC key mkey is uniform and independent of the round keys: this is exactly what the additional, state-separated squeeze in the implementation is designed to deliver under the PRF model, and it is what makes the next reduction sound.

Reduction to KMAC unforgeability. We construct B_{mac} that simulates G_1 for A while interacting with a KMAC oracle keyed by a hidden uniform key, which plays the role of mkey :

1. At the start, B_{mac} samples the round keys $RK_0, \dots, RK_R$ uniformly itself. In G_1 these are independent of mkey , so this sampling is distributed identically to the experiment and requires no knowledge of the MAC key.
2. For each encryption query (N, A, P) from A , B_{mac} generates the keystream by evaluating the Rijndael permutation under its self-sampled round keys on the counter sequence determined by N , and computes $C = P \oplus KS$ blockwise.
3. It forms the MAC input $X = \tau(A, N, C)$ exactly as defined in Section 4.2, queries its KMAC oracle on X to obtain T , and returns (C, T) to A .

The simulation is perfect: the keystream is generated exactly as in G_1 , and tag computation is delegated to the oracle holding the independent MAC key.

Extracting a Forgery. Eventually A outputs a candidate forgery (N^*, A^*, C^*, T^*) satisfying the INT CTXT winning conditions: (C^*, T^*) was not output by any encryption query, and decryption accepts. Verification computes

$$X^* = \tau(A^*, N^*, C^*)$$

and accepts if and only if $\text{KMAC}(\text{mkey}, X^*) = T^*$. If A succeeds, B_{mac} outputs (X^*, T^*) as its MAC forgery. It remains to argue freshness of X^* . Suppose X^* equals some $X_j = \tau(A_j, N_j, C_j)$ queried during the simulation of encryption query j . By Lemma 6.2, transcript equality implies $(A^*, N^*, C^*) = (A_j, N_j, C_j)$, and since KMAC is deterministic the corresponding tag satisfies $T_j = \text{KMAC}(\text{mkey}, X_j) = \text{KMAC}(\text{mkey}, X^*) = T^*$. But then $(C^*, T^*) = (C_j, T_j)$ was returned by encryption query j , contradicting the winning conditions. Hence X^* was never queried to the MAC oracle, and (X^*, T^*) is a valid EUF-CMA forgery whenever A wins. Thus

$$\Pr[G_1(A) = 1] \leq \text{Adv}_{\text{KMAC}}^{\text{euf-cma}}(B_{\text{mac}}),$$

and combining with the $G_0 \rightarrow G_1$ transition gives the stated bound. $\square$

6.3 AEAD Security (IND-CCA)

We now argue that the combination of IND CPA confidentiality and INT CTXT integrity yields IND CCA authenticated encryption for RCS, provided that the decryption oracle

never returns plaintext on invalid tags. This follows the standard Encrypt then MAC composition paradigm of Bellare and Namprempre [11]. The composition requirement — that decryption rejects before revealing any plaintext when tag verification fails — is satisfied by the single-call AEAD interface defined in Section 4, which verifies the tag before invoking the counter-mode transform. The theorems of this section are stated only for that specified interface; internal chunked helpers are outside the RCS formal model.

Theorem 6.4 (IND CCA security of RCS). Suppose RCS is IND CPA secure and INT CTXT secure as established above. Then for any interval-respecting IND CCA adversary A against RCS there exist adversaries B_{cpa} and B_{int} such that

$$\text{Adv}_{\text{RCS}}^{\text{ind-cca}}(A) \leq \text{Adv}_{\text{RCS}}^{\text{ind-cpa}}(B_{\text{cpa}}) + \text{Adv}_{\text{RCS}}^{\text{int-ctxt}}(B_{\text{int}}),$$

and hence, by Theorems 6.1 and 6.3,

$$\text{Adv}_{\text{RCS}}^{\text{ind-cca}}(A) \leq 2 \text{Adv}_{\text{cSHAKE}}^{\text{prf}} + \text{Adv}_{\text{Rijndael}}^{\text{prp}} + \text{Adv}_{\text{KMAC}}^{\text{euf-cma}} + \frac{\sigma^2}{2^{257}}.$$

Proof sketch. The result is an application of the generic theorem that IND CPA security combined with ciphertext integrity yields IND CCA security for Encrypt then MAC schemes, under the condition that decryption rejects before revealing any plaintext when tag verification fails.

We outline the reduction. Let A be an IND CCA adversary for RCS. We construct B_{cpa} that runs A and simulates the IND CCA environment using only access to an IND CPA encryption oracle and an internal INT CTXT adversary B_{int} .

- The encryption oracle in the IND CCA game is simulated directly by the IND CPA oracle, since both return real ciphertexts and tags.
- The decryption oracle must reject any query whose tag does not verify. Any decryption query that would be accepted corresponds to a valid ciphertext tag pair. If such a pair did not come from the encryption oracle, it would constitute an INT CTXT forgery.

Thus either A distinguishes the IND CCA game from IND CPA, which yields a distinguishing advantage for B_{cpa} , or A manages to exploit the decryption oracle in a way that implies an INT CTXT forgery, yielding advantage to B_{int} . Combining these cases gives the inequality above. $\square$

6.4 Key-Commitment Considerations

Some AEAD schemes built on polynomial MACs, including AES-GCM and ChaCha20-Poly1305, are not key-committing: in some settings it is possible to construct or exploit ciphertext–tag pairs that verify under more than one key, which can support partitioning-oracle attacks in password-based or otherwise key-ambiguous protocols [16, 17].

RCS is expected to provide substantially stronger generic key-commitment behavior than polynomial-MAC AEAD schemes because the tag is computed as $T = \text{KMAC}(\text{mkey}, \tau(A, N, C))$, where mkey is derived from the master key through cSHAKE and the transcript encoding binds the associated data, nonce, ciphertext, and final length field. In a generic model that treats KMAC as a random function of the key and transcript, producing a single transcript and tag that verify under two independently derived MAC keys requires a cross-key collision in the keyed sponge output. The generic cost is therefore governed by the tag length: approximately 2^{128} birthday work for RCS-256’s 32-byte tag and 2^{256} birthday work for RCS-512’s 64-byte tag.

This section is a generic-model argument rather than a separate formal key-commitment theorem. A complete treatment would require an explicit key-commitment security experiment and a proof for the cSHAKE-to-KMAC key schedule. The main theorems of this paper are the IND-CPA, INT-CTXT, and IND-CCA results of Sections 6.1–6.3.

6.5 Multi-User Security

In large deployments an adversary may attack many independently keyed instances simultaneously, and generic multi-target tradeoffs can reduce effective security relative to the single-user setting [18]. Two properties of RCS limit this degradation. First, the key sizes of 256 and 512 bits leave exhaustive-search margins of $2^{256-\log_2 u}$ and $2^{512-\log_2 u}$ against u users even under naive multi-target key search, which remains far above practical reach for any realistic user population, and remains so under the Grover-accelerated bounds of Section 6.6. Second, the random 256-bit nonce recommended for general use enters the authenticated transcript and determines the counter interval, so multi-user keystream and tag collisions inherit the same birthday bounds over a 256-bit space as in the single-user analysis. A fully quantitative multi-user treatment, in the style of the analyses performed for AES-GCM in TLS 1.3 [18], is left as future work (Section 10.2); the generic bounds above indicate that no multi-user weakness is introduced by the construction itself.

6.6 Post-Quantum Considerations

This subsection records the post-quantum interpretation of the bounds above. It is not a full quantum-query proof for the composed AEAD construction. In the conservative black-box model, exhaustive search against a λ -bit symmetric key admits Grover-type square-root speedups. This motivates interpreting RCS-256 as targeting approximately 128-bit post-quantum brute-force resistance and RCS-512 as targeting approximately 256-bit resistance.

For the sponge components, the relevant generic bounds are governed by the capacity of the Keccak construction and by the output length of the tag. The KMAC authentication layer uses a 512-bit-capacity sponge and outputs 32-byte or 64-byte tags. These parameters provide a larger generic authentication margin than 128-bit polynomial MAC tags. This distinction is especially relevant in strong quantum-access models: GMAC/GCM is known to be insecure in the Q2 superposition-query model, and Poly1305 has published Q2 attacks below its classical 128-bit target. RCS avoids those polynomial-MAC structures, but a full superposition-query proof for the complete construction is outside the scope of this paper.

The classical bounds from Theorems 6.1 and 6.3 should therefore be read in the post-quantum setting as black-box security claims subject to the usual quadratic losses for generic key search and generic collision or forgery search. They do not assert resistance to all possible quantum structural attacks on Rijndael, cSHAKE, or KMAC beyond the assumptions stated in Section 3.2.

7 Cryptanalytic Evaluation of RCS

This section surveys the resistance of RCS to classical cryptanalytic techniques applied to the underlying wide-block Rijndael permutation and to the sponge-based components used for key expansion and authentication. The goal is not to provide new attacks but to situate the chosen parameters within the landscape of known results, and to justify that the 22-round (RCS-256) and 30-round (RCS-512) configurations sit comfortably beyond current analytical frontiers.

7.1 Wide-Block Rijndael Structure

The core permutation of RCS is a 256-bit Rijndael instance operating on two independent 128-bit halves. Each half is arranged as a 4×4 byte array, and each round applies the standard sequence of Rijndael operations: byte-wise S-box substitution, row rotations, column mixing over $\text{GF}(2^8)$, and addition of a 256-bit round key derived from cSHAKE. The two halves are processed in lockstep, and the round keys span the full 256-bit state.

Relative to AES, which fixes a 128-bit state and uses 10, 12, or 14 rounds, Rijndael-256 increases the diffusion radius and the size of the permutation space. The wider state raises the birthday bound for generic attacks and alters the structure of possible differential and linear trails. The minimum number of active S-boxes over multi-round trails for the large-block Rijndael variants differs from the well-known four-round AES bound, because the ShiftRows offsets and diffusion pattern change with the state width; concrete bounds for Rijndael-256 have been established by dedicated and automated trail analyses in the cryptanalytic literature [14, 3], and it is these large-block bounds, rather than the AES figures, that inform the margins claimed below.

In RCS, the 256-bit permutation is used exclusively in counter-mode: the state input for each block is a counter derived from the 32-byte nonce and an internal block index, and the output is treated as keystream. This usage pattern eliminates complex feedback paths that appear in certain block-cipher modes and simplifies the analysis to that of a keystream generator built from a wide-block PRP.

7.2 Differential and Linear Cryptanalysis

Differential and linear cryptanalysis measure how input differences and linear approximations propagate across the S-box and linear layers of the permutation. The security of Rijndael-256 under these techniques is governed by the number of active S-boxes and the probability of the best differential or linear trails.

Existing analyses of Rijndael-type ciphers show that:

- nontrivial differentials and linear approximations are confined to significantly reduced round counts,
- the minimum number of active S-boxes grows rapidly with the number of rounds, leading to exponential decay in trail probabilities,
- the transition from reduced-round distinguishers to full-round attacks requires a substantial increase in complexity that quickly exceeds practical limits.

RCS sets the round counts to

$$R_{256} = 22, \quad R_{512} = 30,$$

which are substantially higher than the reduced-round regimes targeted by published attacks on Rijndael-256 and AES-like instances. The strongest published single-key results against Rijndael-256 are impossible-differential attacks reaching on the order of 10 to 12 rounds [14], and the strongest results against AES-256 in the related-key model — the full 14-round related-key boomerang attacks of Biryukov and Khovratovich [12] and the practical-complexity attacks on up to 10 rounds of Biryukov et al. [13] — depend essentially on the linear recurrences of the classical AES key schedule, which RCS removes (Section 7.4). The design rationale is to keep the best known differential and linear distinguishers at a distance of many rounds from the operational configurations, and to ensure that any extension of existing trails would require work factors comparable to or greater than exhaustive key search.

From the perspective of the stream cipher usage, the relevant question is whether an adversary can distinguish the keystream from random by exploiting correlations in the underlying permutation. With 22 and 30 rounds and the widened state, the probability of mounting such a distinguisher with complexity below 2^λ is negligible for the intended parameters.

7.3 Algebraic and Structural Attacks

Algebraic cryptanalysis aims to represent the cipher as a system of Boolean or multivariate equations and to solve or approximate this system more efficiently than brute force. For SPN ciphers like Rijndael, two aspects are especially relevant: the algebraic degree of the round functions and the growth of that degree as rounds are iterated.

For Rijndael-256, the nonlinear S-box and linear mixing layer cause the algebraic degree of the overall permutation to grow quickly with each round. After a modest number of rounds the algebraic degree approaches its maximum, and the resulting systems for full-round instances become too complex for current equation-solving techniques to exploit in practice.

Structural attacks such as slide attacks, meet-in-the-middle techniques, and invariant subspace attacks typically require regularities in the round structure or key schedule. In RCS, each round key is derived independently from the cSHAKE-based key schedule, which breaks the kind of linear recurrences and self-similar patterns that underlie many structural cryptanalytic constructions.

The choice of 22 and 30 rounds ensures that any algebraic representation of the full permutation would involve a very high degree and a large number of variables, placing known algebraic methods well beyond feasible bounds for both parameter sets.

7.4 Related-Key and Key-Schedule Attacks

Classical Rijndael key schedules are known to admit related-key patterns and structures that enable certain attacks on reduced-round variants, and in the case of AES-256, on the full cipher: the related-key boomerang attacks of Biryukov and Khovratovich [12] break all 14 rounds of AES-256 with complexity around $2^{99.5}$, and related techniques achieve practical complexity on up to 10 rounds [13]. These attacks rely on linear recurrences in the key expansion, which create predictable relations between round keys derived from related master keys; they are the strongest evidence available that the classical schedule, not the round function, is the weakest element of the AES design, and they motivate the RCS key schedule directly.

RCS replaces the classical key schedule with a cSHAKE-based expansion that treats the master key, optional information string, and a fixed domain-separation string as input to a Keccak sponge. The resulting output stream is then sliced into a MAC key and a sequence of independent 256-bit round keys. Under the PRF assumption for cSHAKE, the round keys are indistinguishable from independently uniform random values, even when master keys are related by efficiently computable transformations.

From the point of view of related-key security, any adversary that exploits relations between round keys across different master keys would induce a distinguishing attack on cSHAKE. The formal model in Section 5 therefore treats related-key attacks on the permutation as subsumed by the PRF security of the key-expansion function. Within this model, standard related-key and slide attacks against Rijndael do not apply to RCS.

7.5 The Reduced-Rounds Configuration

The optional reduced-rounds build (Section 2.1) lowers the permutation round counts to 14 (RCS-256) and 21 (RCS-512) and replaces KMAC-R24 with KMAC-R12, halving the

Keccak- f [1600] round count in the authentication layer. This configuration is excluded from the formal claims of Section 6 and from the margins of Table 4; we assess it briefly here.

Permutation Margin. Against the best published single-key analyses of Rijndael-256, which reach on the order of 10 to 12 rounds [14], the 14-round reduced RCS-256 retains a positive but substantially thinner margin of roughly 2 to 4 rounds, compared to 9 or more rounds for the standard 22-round configuration. The 21-round reduced RCS-512 retains a margin of roughly 9 rounds. Two mitigating factors apply to both: the published attacks are key-recovery and distinguishing results against the block cipher in chosen-plaintext settings, whereas RCS exposes the permutation only through counter-mode keystream on adversary-visible but not adversary-chosen inputs; and the attacks that penetrate deepest into AES-style designs exploit the classical key schedule, which is absent here. Nevertheless, a 2-to-4 round margin leaves little allowance for future cryptanalytic improvement, and the reduced RCS-256 permutation should be regarded as suitable only for the short-exposure scenarios for which the option is intended.

Authentication Margin. KMAC-R12 applies 12 of the 24 Keccak- f [1600] rounds per permutation call, with the sponge capacity unchanged at $c = 512$ bits. Published practical collision attacks on Keccak-based hash instances reach approximately 6 rounds, with theoretical preimage and distinguishing results extending somewhat further; no published forgery or key-recovery attack approaches 12 rounds of the keyed sponge, where the secret key and the 512-bit capacity deny the attacker the known-state starting point that the hash-setting attacks require. The 12-round keyed-MAC margin is therefore positive against all published results, while clearly less conservative than the full-round instance. The distinct `KMACR12` name string domain-separates the reduced MAC from standard KMAC, so tags produced by one mode are never valid in the other.

Scope. The hybrid arguments of Section 6 are agnostic to the round counts and carry over to the reduced configuration *formally*, but the assumptions they rest on — the PRP modeling of the permutation and the PRF/EUF-CMA modeling of the MAC — are supported by materially thinner cryptanalytic evidence at the reduced parameters. Deployments requiring long-term confidentiality, margin against future cryptanalysis, or conservative assurance must use the standard configuration.

7.6 Summary of Cryptanalytic Margins

The cryptanalytic evaluation can be summarized as follows:

- **Differential and linear attacks.** Known distinguishers and key-recovery attacks on Rijndael-type ciphers reach only significantly reduced round counts and do not extend to the 22- and 30-round configurations used by RCS. The widened 256-bit state further increases the diffusion per round and the number of active S-boxes.
- **Algebraic and structural attacks.** Algebraic degree growth and the absence of exploitable structure at the chosen round counts place RCS beyond the reach of current equation-based and structural techniques, given realistic resource bounds.
- **Related-key attacks.** The cSHAKE-based key schedule yields round keys that behave as independent random values under standard assumptions, eliminating the linear relations exploited by classical related-key attacks on Rijndael.
- **Overall margin.** For both RCS-256 and RCS-512, the best known attacks against the underlying permutation remain well below the configured round counts and do

not offer any advantage over generic key search at the intended security levels. Within the adopted models for Rijndael, cSHAKE, and KMAC, RCS therefore maintains conservative cryptanalytic margins for its intended deployment scenarios.

These conclusions are consistent with the reductionist view developed in Section 6: the security of RCS reduces to the PRP behavior of the wide-block Rijndael permutation under independent keys and the PRF and EUF-CMA properties of the Keccak-based components, all of which enjoy substantial analytical support in the existing literature.

Table 3 summarizes the effective security margins of RCS against the primary attack families considered in the cryptanalytic evaluation. The margins are expressed in rounds and represent the gap between the best published reduced-round analyses of Rijndael-256, principally the impossible-differential results of Liu et al. [14] and the structural literature surveyed in [3], and the full-round configurations used in RCS-256 (22 rounds) and RCS-512 (30 rounds). Related-key and slide attacks are marked not applicable because the cSHAKE key schedule removes the key-schedule relations they require (Section 7.4).

Table 4: Cryptanalytic margins for RCS. Best-known figures refer to single-key distinguishers and key-recovery attacks on reduced-round Rijndael-256 reported in the literature [14]; related-key and slide entries are not applicable due to the cSHAKE key schedule.

Attack	Best Known	256 Margin	512 Margin
Differential	≤ 12 –13 rounds	9–10	17–18
Linear	≤ 11 –12 rounds	10–11	18–19
Algebraic	~ 10 rounds	12	20
Related-key	n/a	n/a	n/a
Structural/slide	n/a	n/a	n/a

8 Misuse Resistance and Operational Limits

This section addresses the operational assumptions under which the security guarantees of RCS hold, with particular emphasis on nonce handling in counter mode and limitations that arise when these assumptions are violated. These considerations define the boundary between the provable guarantees established in previous sections and practical misuse scenarios that fall outside the formal model.

8.1 Nonce Reuse and its Consequences

RCS is *not* misuse resistant with respect to nonce reuse. As a counter-mode construction, RCS requires that the 256-bit nonce N be *unique for each encryption under a fixed key*. If a nonce is ever reused with the same key, the confidentiality of both messages encrypted under that nonce collapses immediately.

Let P_1 and P_2 be two plaintexts encrypted under the same key–nonce pair, and let KS_i denote the keystream blocks derived from the fixed counter sequence associated with that nonce. The ciphertexts satisfy

$$C_1 = P_1 \oplus KS, \quad C_2 = P_2 \oplus KS,$$

implying

$$C_1 \oplus C_2 = P_1 \oplus P_2.$$

Thus an adversary observing both ciphertexts learns the XOR of the plaintexts, which

generally suffices to recover both messages whenever either plaintext possesses known structure or redundancy. This attack is independent of the number of rounds of the permutation and does not rely on any weakness in Rijndael or cSHAKE.

Nonce reuse therefore constitutes a *catastrophic* confidentiality failure. RCS must be deployed with strict nonce management, ensuring that no nonce–key pair is ever repeated. This requirement is explicitly assumed in all security definitions and reductions.

8.2 Counter-Interval Overlap and Sequential Nonces

Because RCS uses the nonce directly as the initial counter value, a second, subtler misuse condition exists alongside literal nonce reuse: *counter-interval overlap*. A message of m blocks encrypted under nonce N consumes the counter values $N, N + 1, \dots, N + m - 1$. If a later message is encrypted under any nonce N' falling inside that interval — for example $N' = N + 1$ — then the two messages share keystream blocks even though the nonces are distinct, and the consequences are identical to those of nonce reuse described above: the XOR of the overlapping plaintext blocks is revealed.

For this reason, naive sequential nonces $(N, N + 1, N + 2, \dots)$ *must not* be used with RCS, in contrast to constructions that pair a short counter with a separate per-message nonce field. The supported nonce strategies are:

- **Random nonces (general use).** Nonces are generated uniformly at random by a CSPRNG. The probability that two random 256-bit intervals of combined length σ blocks overlap is at most $q_E \sigma / 2^{256}$ (Theorem 6.1), which is negligible for all realistic workloads.
- **Deliberate interval partitioning (parallel processing).** This is the primary motivation for exposing the full 256-bit counter as the nonce: a large data set can be encrypted in parallel by assigning each worker a nonce offset at the correct block boundary, so that the workers' counter intervals partition the data with no overlap by construction. The caller is responsible for the correctness of the partition.
- **Interval-spaced sequential nonces.** If a stateful sequential scheme is required, successive nonces must be advanced by at least the maximum number of blocks in any message (plus the per-call nonce consumption), so that consecutive intervals cannot intersect.

In all cases the requirement is the interval-disjointness condition of Section 3.3; nonce uniqueness is necessary but not sufficient.

8.3 Scope of the Single-Call Interface

The formal guarantees of Section 6 apply to the single-call AEAD interface specified in this paper. That interface verifies the authentication tag before releasing plaintext on decryption and is the only RCS transform included in the formal specification. Internal chunked helpers used by other protocols are outside the RCS specification and are not modeled here.

8.4 Modeling Assumptions and Out-of-Scope Attacks

The formal analysis of RCS relies on a number of explicit modeling assumptions that bound the adversarial capabilities considered. The following scenarios fall outside the guarantees provided by the proofs:

Incorrect or Predictable Nonce Generation. If nonces are generated deterministically without safeguarding both uniqueness and counter-interval disjointness (Section 8.2),

or if an adversary can predict or influence nonce values, then the confidentiality guarantees do not hold. RCS does not provide nonce-misuse-resistant encryption, forward secrecy under reuse, or synthetic IV behavior.

Weak or Compromised Keys. The analysis assumes that keys are sampled uniformly from $\{0,1\}^\lambda$ and remain secret for the duration of their use. If the master key is poorly generated, predictable, or exposed to the adversary, the security guarantees are void.

Side-Channel Leakage. The proofs treat all underlying primitives as black-box idealizations and do not cover timing, power, cache, electromagnetic, or fault-injection side channels. Although the reference implementation is written to avoid obvious timing leaks, formal side-channel resistance is out of scope.

State Rollback or Incomplete Zeroization. The model assumes correct implementation behavior in which the internal Keccak state, round keys, counters, and MAC keys are destroyed at the end of an operation. If an implementation fails to erase state or allows state rollback (e.g., via persistence or virtualization artifacts), confidentiality or integrity may be compromised.

Associated-Data Misuse. Associated data is authenticated but not encrypted. The model assumes that the caller supplies all associated data before encryption or decryption, and that the associated data is treated consistently across both operations. Misuse of associated data invariants is outside the formal guarantees.

Out-of-Scope Cryptanalytic Models. The analysis does not address:

- attacks assuming related-message structures across multiple keys,
- multi-user or multi-target amplification attacks beyond standard bounds,
- attacks exploiting physical co-location or shared hardware behavior,
- deliberate weakening of cSHAKE customization parameters.

Within the formal scope defined by the AEAD syntax, unique nonce usage, correct implementation of cSHAKE and KMAC domain separation, and the PRP/PRF properties of the underlying primitives, the RCS construction achieves the confidentiality and integrity guarantees established in the preceding sections. The conditions listed here specify the assumptions that must be upheld by any correct implementation or deployment of RCS.

9 Implementation Conformance and Side Channels

This section relates the formal RCS model to the reference implementation and records the assumptions required for side channel resistance and randomness quality. The intent is to ensure that the deployed code actually realizes the scheme analyzed in the previous sections.

9.1 Mapping Between Model and Reference Code

The public interface in `rcs.h` corresponds directly to the AEAD syntax

$$(K, N, A, P) \mapsto (C, T)$$

formalized in Sections 3 and 4. The main structures and functions align with the model as follows.

State and Parameters. The structure `qsc_rcs_state` contains:

- the cipher type flag, which selects RCS-256 or RCS-512 and fixes the key size, tag size and round count (22/30 in the standard build, 14/21 in the reduced-rounds build of Section 2.1),
- the round key array, which stores $(R + 1)$ 256-bit round keys used by the Rijndael permutation,
- the Keccak sponge state used for cSHAKE and KMAC operations,
- the stored nonce, a 32 byte buffer that represents N and is advanced in place by the counter-mode layer to produce successive counter blocks,
- an internal byte counter, initialized to 1, that accumulates the message length and feeds only the final length encoding absorbed by the MAC; it does not drive the counter blocks,
- mode flags that indicate encryption or decryption.

These fields implement the abstract state variables specified in the formal definition of RCS.

Initialization and Key Expansion. The function `qsc_rcs_initialize` instantiates the scheme for a given key, nonce and optional information string. It performs the following steps:

1. chooses the correct parameter set (RCS-256 or RCS-512) based on the provided key size,
2. initializes the cSHAKE instance with the master key, the optional info tweak string, and the fixed RCS domain string,
3. squeezes the sequence of round keys from the cSHAKE output stream, and then, from a further state-separated squeeze, the MAC key (Section 2.4),
4. copies the nonce into the state and initializes the internal byte counter to 1.

This implements the key expansion procedure of Section 4.2, in which

$$(\text{mkey}, RK_0, \dots, RK_R)$$

are derived from cSHAKE under a fixed domain string.

Transform Functions. The function `qsc_rcs_transform` implements the encryption and decryption algorithms described in Section 4. Internal chunked helpers, including `qsc_rcs_extended_transform`, are outside the formal RCS specification.

The single-call transform:

- produces counter blocks by advancing the stored nonce buffer in place,
- applies the Rijndael-256 round function to obtain keystream blocks,
- XORs keystream blocks with input blocks to produce output blocks,
- absorbs the nonce, the ciphertext, and the final length encoding into the KMAC state (after any associated data absorbed via `qsc_rcs_set_associated`),
- outputs or verifies the authentication tag.

When the mode flag selects encryption, the function produces (C, T) from P . When the mode flag selects decryption, the function recomputes the tag, compares it in constant time, and invokes the counter-mode transform only after the comparison succeeds; on failure no plaintext is produced. This behavior matches the Encrypt then MAC model required by the IND-CCA reduction. Internal chunked helpers are excluded from this formal specification.

Transcript Correspondence. The abstract transcript $\tau(A, N, C)$ of Section 4.2 is realized in the reference code as the exact byte sequence absorbed into the KMAC state by the calls `qsc_rcs_set_associated` (which absorbs A followed by $\text{le}_{32}(|A|)$), the nonce and ciphertext updates performed by `rsc_mac_update` inside `qsc_rcs_transform` (which absorb N and then C), and `rsc_mac_finalize` (which absorbs $\text{le}_{64}(32 + \text{ctr} + 8) = \text{le}_{64}(41 + |C|)$ before extracting the tag). The injectivity property of Lemma 6.2 therefore applies verbatim to the implemented absorption sequence.

9.2 Constant Time Behavior and Leakage

The security proofs treat RCS as an idealized black box. To approach this model in practice, the implementation must avoid control flow and memory access patterns that depend on secret data.

At a minimum, the following conditions should hold:

- **Key schedule.** The cSHAKE-based key expansion runs on fixed length inputs and uses only public loop bounds. The number of calls to the Keccak permutation and the memory access pattern within the sponge do not depend on the key bits.
- **Permutation rounds.** The Rijndael-256 permutation applies a fixed sequence of S-box lookups, row shifts and column mixes for each round. The round count and order of operations are independent of key and data values.
- **Counter mode.** Counter increment operations act on public counters and do not branch based on secret bits. The mapping from block index to counter value is deterministic and public. The scalar and single-block AES-NI paths propagate carries across the full 32-byte counter (`qsc_intutils_le8increment`), implementing the $(N + i) \bmod 2^{256}$ arithmetic of the model exactly. The AVX-512 fast path increments only the low 64-bit lane of each counter block; its output is identical to the carry-propagating paths under the precondition that the low 64-bit lane does not wrap within the call, i.e., $\text{le}_{64}(N[0..7]) + \sigma_{\text{msg}} \leq 2^{64} - 1$, where σ_{msg} is the number of blocks processed. For random nonces this precondition fails only with probability approximately $\sigma_{\text{msg}}/2^{64}$ per message; implementations should nevertheless enforce it explicitly with a public-data guard that falls back to the carry-propagating path when the lane would wrap, ensuring bitwise-identical output across all code paths for all nonce values. The guard branches only on public counter data and introduces no secret-dependent timing variation.
- **Tag comparison.** Tag verification uses a constant time comparison that scans every byte and aggregates differences without early exit. No timing variation should reveal whether a prefix of the tag matches.
- **Error handling.** On tag failure, the implementation should avoid emitting partial plaintext or performing data dependent cleanup. The time to reject invalid tags should depend only on public parameters such as message length.

These properties do not guarantee resistance to all microarchitectural attacks, but they are necessary preconditions for any realistic constant time implementation. Deployment

in high-assurance environments may require additional hardening against cache, branch predictor and speculative execution side channels.

Conformance Test Vectors. Known-answer tests should include a counter-boundary vector using a nonce whose low 64-bit lane lies within a few blocks of $2^{64} - 1$, exercising the carry path and confirming that vectorized and scalar implementations produce identical output across the lane boundary.

9.3 Random Number Generation Requirements

The formal model assumes that both keys and nonces are generated in a way that prevents adversarial prediction and reuse. This implies the following operational requirements.

Key Generation. Master keys must be generated from a cryptographically secure random number generator that provides at least λ bits of entropy. Keys must not be derived from passwords or other low entropy sources unless they are first processed by a suitable key derivation function whose security matches or exceeds that of RCS.

Nonce Generation and Management. Nonces must be unique for each encryption under a given key, and the counter intervals they induce must be pairwise disjoint (Section 8.2). Acceptable strategies are:

- a random nonce generated by a CSPRNG for each encryption, for which both repetition and interval overlap occur only with negligible probability at the 256-bit nonce size; this is the recommended general strategy,
- deliberate interval partitioning for parallel processing, in which worker nonces are offset at block boundaries so that their counter intervals partition the data without overlap by construction,
- a stateful per-key counter, stored in non-volatile memory and updated atomically, that advances by at least the maximum number of blocks per message between encryptions, so that consecutive intervals cannot intersect; a counter that advances by one per message must *not* be used, as it guarantees interval overlap for multi-block messages.

Reusing a nonce under the same key, or encrypting under nonces with overlapping counter intervals, causes keystream reuse, which breaks confidentiality as discussed in Section 8. Implementations must therefore treat nonce uniqueness and interval disjointness as hard requirements, not as best effort properties.

Associated Randomness. If RCS is composed with higher level protocols that derive keys or nonces from other primitives (such as KDFs or key exchange mechanisms), those components must provide security at least comparable to the RCS parameter set in use. The formal results in this paper do not cover failures in upstream randomness sources.

Under these conditions, the behavior of the deployed implementation conforms to the assumptions made in the provable security analysis.

10 Conclusion

The analysis presented in this paper establishes a rigorous and implementation aligned security foundation for the RCS authenticated cipher stream. By connecting the engineering specification directly to a formal model and by applying standard game based reasoning,

we obtain confidentiality and integrity guarantees that reduce cleanly to the well studied properties of the underlying Rijndael and Keccak primitives.

10.1 Summary of Results

We summarize the principal contributions and findings:

- **Formalization of the Scheme.** The RCS construction was defined precisely through an engineering level specification and a mathematical abstraction that captures the behavior of the reference implementation. This provides a single authoritative model for analysis and future standardization.
- **Confidentiality Proof.** Through a four step hybrid argument, we reduced IND CPA security of RCS to the PRF security of cSHAKE and the PRP security of the 256 bit Rijndael permutation under independently derived round keys, with an explicit PRP-PRF switching term. Under unique, interval-disjoint nonce usage, the resulting keystream is indistinguishable from uniformly random strings.
- **Integrity Proof.** Ciphertext integrity was reduced to the PRF security of the cSHAKE key derivation together with the EUF-CMA security of the KMAC function, applied to an injectively encoded transcript comprising the associated data and its length, the nonce, the ciphertext, and the final encoded length field. An explicit injectivity lemma ensures that any successful forgery against RCS yields a forgery against KMAC.
- **AEAD Composition.** By combining confidentiality and integrity, and using the fact that the single-call interface rejects invalid tags before releasing plaintext, we proved IND CCA security for the AEAD construction following the Encrypt then MAC paradigm [11].
- **Key-Commitment and Multi-User Considerations.** We gave a generic-model argument that the sponge-based authentication layer provides stronger key-commitment behavior than polynomial MAC schemes in key-ambiguous settings, and summarized the construction’s multi-user considerations.
- **Cryptanalytic Margins.** We reviewed differential, linear, algebraic and related key attacks on Rijndael type ciphers, and concluded that the 22 round (RCS 256) and 30 round (RCS 512) configurations remain well beyond the reach of published techniques. The use of cSHAKE-derived independent round keys eliminates key schedule based attacks.
- **Post-Quantum Considerations.** We argued that the classical bounds degrade gracefully under quantum query models, preserving roughly 2^{128} and 2^{256} security for RCS 256 and RCS 512, respectively, in line with the usual square root attack heuristics.

Together, these results show that RCS is a conservative, transparent and well structured AEAD primitive whose security reduces to well understood assumptions on widely analyzed cryptographic components.

10.2 Limitations and Future Work

Although the analysis covers the core security claims of the RCS design, certain limitations remain and motivate possible refinement:

- **Nonce Misuse.** RCS does not provide misuse resistance under nonce reuse or counter-interval overlap. While this is inherent to counter-mode designs that ex-

pose the full counter as the nonce, formalizing RCS variants with synthetic IV or deterministic nonce generation could mitigate operational risks.

- **Chunked Operation.** The formal guarantees cover the single-call AEAD interface. Internal chunked helpers used by other protocols are excluded from the RCS specification. A formally analyzed segmented streaming mode with per-segment authentication, in the style of STREAM [15], would be a separate construction.
- **Side Channel Resistance.** The proofs treat the permutation and sponge components as idealized black boxes. Further work could include formal leakage models, masked implementations, or provable bounds under bounded leakage assumptions.
- **Reduced Round Analysis.** Although the chosen round counts of the standard configuration lie well above known cryptanalytic thresholds, exploring higher fidelity bounds, automated trail search, and algebraic degree analyses specific to the 256 bit Rijndael variant could provide quantitative insight into long term safety margins. Such analysis would be particularly valuable for the optional reduced-rounds build (Sections 2.1 and 7.5), whose 14-round RCS-256 permutation carries a margin of only a few rounds over the best published attacks and which is currently assessed by margin arguments rather than by the formal claims of Section 6.
- **Quantum Adversary Models.** A more complete treatment of cSHAKE and Rijndael under quantum superposition queries, or the implications of quantum chosen ciphertext attacks, may refine the understanding of post-quantum behavior beyond the square root heuristic.
- **Multi-User and Compositional Security.** Extending the qualitative multi-user discussion of Section 6.6 to fully quantitative bounds, and treating key rotation processes or compositional frameworks for large scale systems, may support integration of RCS into complex protocols.

RCS shows that combining a wide-block Rijndael permutation with Keccak-based key derivation and authentication yields a simple and analyzable AEAD design under explicit nonce, implementation, and primitive assumptions. Further standardization work, conformance testing, and cryptanalytic review would strengthen its suitability for high-assurance deployment.

References

- [1] QRCS Corporation. *RCS Technical Specification* Quantum Resistant Cryptographic Solutions Corporation, 2024. Available at: https://www.qrcscorp.ca/documents/rcs_specification.pdf
- [2] QRCS Corporation. QSC Cryptographic Library Implementation (GitHub Source Code Repository). <https://github.com/QRCS-CORP/QSC>
- [3] Daemen, J., Rijmen, V. *The Design of Rijndael: AES, the Advanced Encryption Standard*. Springer, 2002. Available at: <https://doi.org/10.1007/978-3-662-04722-4>
- [4] Daemen, J., Mennink, B., Van Assche, G. *The Keccak Reference*. Submission to the NIST SHA3 Competition, 2011. Available at: <https://keccak.team/files/Keccak-reference-3.0.pdf>
- [5] Bertoni, G., Daemen, J., Peeters, M., Van Assche, G. *Keccak Specifications and Implementations*. NIST, 2012. Available at: <https://keccak.team/files/Keccak-specifications.pdf>
- [6] National Institute of Standards and Technology (NIST). *FIPS 197: Advanced Encryption Standard (AES)*. U.S. Department of Commerce, 2001. Available at: <https://doi.org/10.6028/NIST.FIPS.197>
- [7] National Institute of Standards and Technology (NIST). *FIPS 202: SHA3 Standard*. U.S. Department of Commerce, 2015. Available at: <https://doi.org/10.6028/NIST.FIPS.202>
- [8] National Institute of Standards and Technology (NIST). *SP 800-185: SHA3 Derived Functions, cSHAKE, KMAC, TupleHash, and ParallelHash*. U.S. Department of Commerce, 2016. Available at: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-185.pdf>
- [9] Daemen, J., Rijmen, V. *Rijndael Reference Code and Test Vectors*. AES Submission Archive. Available at: <https://csrc.nist.gov/Projects/block-cipher-techniques/aes-development> Direct package: <https://csrc.nist.gov/csrc/media/projects/block-cipher-techniques/documents/aes-development/aes-rijndael.pdf>
- [10] Keccak Team. *Keccak Reference Software and Documentation*. Available at: <https://keccak.team/>
- [11] Bellare, M., Namprempre, C. *Authenticated Encryption: Relations Among Notions and Analysis of the Generic Composition Paradigm*. Journal of Cryptology 21(4), 2008. Available at: <https://eprint.iacr.org/2000/025>
- [12] Biryukov, A., Khovratovich, D. *Related-Key Cryptanalysis of the Full AES-192 and AES-256*. ASIACRYPT 2009, LNCS 5912, Springer. Available at: <https://eprint.iacr.org/2009/317>
- [13] Biryukov, A., Dunkelman, O., Keller, N., Khovratovich, D., Shamir, A. *Key Recovery Attacks of Practical Complexity on AES-256 Variants with up to 10 Rounds*. EUROCRYPT 2010, LNCS 6110, Springer. Available at: <https://iacr.org/archive/eurocrypt2010/66320198/66320198.pdf>
- [14] Liu, Y., et al. *Improved Impossible Differential Cryptanalysis of Large-Block Rijndael*. Science China Information Sciences 62(3), 2019. Available at: <https://doi.org/10.1007/s11432-017-9365-4>

- [15] Hoang, V.T., Reyhanitabar, R., Rogaway, P., Vizár, D. *Online Authenticated-Encryption and its Nonce-Reuse Misuse-Resistance*. CRYPTO 2015, LNCS 9215, Springer. Available at: <https://eprint.iacr.org/2015/189>
- [16] Len, J., Grubbs, P., Ristenpart, T. *Partitioning Oracle Attacks*. USENIX Security Symposium, 2021. Available at: <https://www.usenix.org/conference/usenixsecurity21/presentation/len>
- [17] Albertini, A., Duong, T., Gueron, S., Kölbl, S., Luykx, A., Schmieg, S. *How to Abuse and Fix Authenticated Encryption Without Key Commitment*. USENIX Security Symposium, 2022. Available at: <https://www.usenix.org/conference/usenixsecurity22/presentation/albertini>
- [18] Bellare, M., Tackmann, B. *The Multi-User Security of Authenticated Encryption: AES-GCM in TLS 1.3*. CRYPTO 2016, LNCS 9814, Springer. Available at: <https://eprint.iacr.org/2016/564>